Deliverable 1's Report

Alberto Cimmino
Università di Trento

***Abstract*—This paper analyses multiple CPU and GPU implementations of Sparse Matrix-Vector Multiplication (SpMV) using COO and CSR sparse formats. Different size and shape matrices are used to compare different memory-access strategies and GPU optimizations, highlighting the importance of synchronization, load balancing and parallelization.**

***Index Terms*—SpMV, Storage Formats, COO, CSR, CUDA**

I. Introduction

Sparse matrix-vector multiplication (SpMV) is a common operation in many scientific applications. As the name suggests, SpMV consists in the multiplication of a sparse matrix – a matrix which elements are mostly zero – and a dense vector. SpMV is known to be a bottleneck in scientific computational applications and to not use efficiently the processor performance [3], making it memory bound due to its irregular access to memory. [1]

The goal of this paper is to describe and analyse various implementations to solve the SpMV problem, both on GPU and CPU. Different kernels will be explored and their performances will be analysed with regard to the matrix shape, highlighting the benefits of each kernel.

A. Problem Statement

The SpMV problem is formally defined by Equation 1, where A is a sparse matrix $m \times n$, v is a dense vector $n \times 1$ and y is a dense vector of shape $m \times 1$.

$$y = Av \tag{1}$$

Let us define nnz as the number of non-zero elements in A. Since A is defined to be a sparse matrix, we know that $nnz \ll (n \times m)$, and as such, storing the full $n \times m$ matrix would be vastly inefficient, both for memory space and computational time. In order to solve these problems different storage formats exist. This paper will focus on the "Coordinate" (COO) format and one of its optimizations: the "Compressed Sparse Row" (CSR) format. The COO storage format stores in memory three arrays, each of size nnz:

- **ARow**: storing the row indexes of the elements which are non-zeroes;
- **ACol**: storing the column indexes of the elements which are non-zeroes;
- **AVal**: storing the actual values.

This format has the benefits of being pretty intuitive and its corresponding SpMV implementation are easy to understand, however it is quite inefficient, having redundant storage accesses in the case that the matrix has many non-zero values in the same row or column [2]. The CSR storage format is an optimized version of the COO format. It does so by compressing the row indices array and keeping the other two the same. Instead of storing each index, it stores the global position of the first non-zero element in each row. This format has two main benefits: the reduction in space-complexity, closer to $O(2nnz)$ instead of $O(3nnz)$ and the additional robustness against data distribution [2].

B. Implementations overview

This paper will explore six GPU algorithms, two will use the COO format and the remaining four will use a CSR storage format. Two CPU algorithms (one for each storage format used) will be used as the baseline for the experiments. The COO algorithms will be run in two different access modes: sequential and strided. The CSR ones, instead, will test 4 different configurations: sequential access, strided access using the global memory, strided access using the shared memory and a final improvement version that attempts to avoid bank conflicts.

II. Methodology

A. CPU Implementation

Two CPU implementations were made, both to check the correctness of the program and as a baseline to compare the GPU algorithms. The main difference between the two is the storage format, which has been explained already and as such requires no additional focus. Other than that, both algorithms use a sequential access pattern, and as such loop over all non-zeroes values to perform the SpMV.

B. COO GPU Implementation

For these implementations, the number of blocks and the number of threads per block is given to the algorithm as a parameter and is not calculated at runtime. These values were chosen as multiple powers of 2, the number of blocks range from 2^1 to 2^{14}, while for threads they range from 2^5 to 2^{10}.

Sequential Access

This is the most straight-forward algorithm of this paper. Each thread processes a contiguous portion of the non-zeroes elements and then compute the corresponding products. Each thread works on $nnz \div total\_threads$ adjacent elements. This leads to a non-coalesced access to memory, as different threads will access non-continuos areas of memory. Furthermore, since different threads may update the same cell of the output vector, $atomicAdd()$ is required in order to avoid race conditions. As the name implies, this operation is atomic and so will slow down the execution. Algorithm 1 shows its implementation.

Strided Access

An improvement on the previous implementation is to have a coalesced access to the memory. This can be achieved by accessing the global memory via a stride. Adjacent threads access adjacent memory locations during each iteration of the loop, enabling coalesced global memory accesses and reducing

Algorithm 1: COO Global Memory SpMV (Sequential Access)

Input: $ARow$, $ACol$, $AVal$, vector v, nnz
Output: $result$

$tid \leftarrow blockIdx.x \times blockDim.x + threadIdx.x$;
$total\_threads \leftarrow gridDim.x \times blockDim.x$;
$nnz\_per\_thread \leftarrow nnz \div total\_threads$;

$start \leftarrow tid \times nnz\_per\_thread$;
$end \leftarrow \min(start + nnz\_per\_thread,\ nnz)$;
for $i \leftarrow start$ **to** $end - 1$ **do**
    $product \leftarrow AVal[i] \times v[ACol[i]]$;
    atomicAdd$(result[ARow[i]],\ product)$;
end

the number of memory transactions. The size of the stride is the same as the total number of threads for each block, then each thread will jump from t_i to $t_i + stride$ as its next element. Algorithm 2 show its implementation.

Algorithm 2: COO Global Memory SpMV (Strided Access)

Input: $ARow$, $ACol$, $AVal$, vector v, nnz
Output: $result$

$tid \leftarrow blockIdx.x \times blockDim.x + threadIdx.x$;
$total\_threads \leftarrow gridDim.x \times blockDim.x$;
$i \leftarrow tid$;
while $i < nnz$ **do**
    $product \leftarrow AVal[i] \times v[ACol[i]]$;
    atomicAdd$(result[ARow[i]],\ product)$;
    $i \mathrel{+}= total\_threads$;
end

C. CSR GPU Implementation

For these implementations, the number of threads per block is given to the algorithm as a parameter, while the number of blocks is calculated at runtime. The values tested for the number of threads were powers of 2, ranging from 2^5 to 2^{10}.

Sequential Access

As stated before, the CSR format is an improvement on the COO storage. Therefore an improvement is to adapt the previously described algorithms to use this format. In addition, since one thread is assigned to each row the call to $atomicAdd$ can be removed. This, however, can lead to some load unbalancing.

Strided Access

Similarly, accessing the data in a strided manner can improve the memory access. The main difference is that, since GPUs execute threads in groups called *warps*, the algorithm will access the data according to the $warpsize$ instead of looking at the total number of threads. This will improve the memory latency by making the threads request nearby memory locations, allowing the GPU to combine multiple memory requests.

Improvements with shared memory

An additional optimization exploits the GPU shared memory to reduce the number of redundant global memory accesses. Intermediate partial sums are accumulated on the shared memory instead of directly updating global memory after each multiplication. This reduces the frequency of expensive global memory operations and allows threads within the same block to compute partial results. A final reduction step is performed to accumulate each thread contributions before writing the result to global memory. This approach improves performance especially for matrices with a high number of non-zero elements per row, where multiple threads contribute to the same output entry. Algorithm 3 show its implementation.

Bank Conflict Improvements

Algorithm 3: CSR Shared Memory SpMV (Strided Access)

Input: $rowPtr$, $lenRP$, $ACol$, $AVal$, vector v, nnz
Output: $result$

$extern\ \_\_shared\_\_\ shared[\ ]$;
$warp\_id \leftarrow threadIdx.x \div 32$;
$lane \leftarrow threadIdx.x\ \ \&\ \ 31$;
$vals[threadIdx.x] \leftarrow 0.0$;
$\_\_syncthreads()$;
if $row < lenRP$ **then**
    $i \leftarrow row + lane$;
    while $i < end$ **do**
        $shared[threadIdx.x] \mathrel{+}= AVal[i] \times v[ACol[i]]$;
        $i \mathrel{+}= warpSize$
    end
end

`/* Intra-Thread Reduction */`;

if $lane \equiv 0$ **then**
    $atomicAdd(result[row], shared[threadIdx.x])$;
end

To further optimize shared memory usage, it is important to avoid bank conflicts, which occur when multiple threads within a warp access different addresses mapped to the same memory bank. These conflicts serialize memory accesses and so reduce the effective bandwidth of shared memory. In order to prevent this, the shared memory layout can be padded or reorganized so that consecutive threads access different memory banks. These optimizations are particularly relevant for SpMV kernels, where irregular access patterns can easily lead to contention in shared memory. Algorithm 4 show its implementation.

D. Environment

Hardware

The experiments were run on the DISI Cluster, which utilizes an NVIDIA A30[1] as the GPU and an AMD EPYC 9334[2] as the CPU .

[1]https://www.nvidia.com/content/dam/en-zz/Solutions/data-center/products/a30-gpu/pdf/a30-datasheet.pdf - Last visited on: May 16, 2026

[2]https://www.amd.com/en/products/processors/server/epyc/4th-generation-9004-and-8004-series/amd-epyc-9334.html - Last visited on: May 16, 2026

Algorithm 4: CSR Shared Memory SpMV (Strided Access), Bank Conflicts

Input: $rowPtr$, $ACol$, $AVal$, vector v, nnz
Output: $result$

```
extern __shared__ shared_data[ ];
tid ← blockIdx.x × blockDim.x + threadIdx.x;
lane ← tid & 31;
row ← blockIdx.x × (blockDim.x ÷ 32) + warp_id;
warp_shared ← &shared_data[warp_id * 32];
warp_shared[lane] ← 0.0;
__syncthreads();

i ← rowPtr[tid ÷ 32] + lane;
while i < rowPtr[tid ÷ 32 + 1] do
    warp_shared[lane] += AVal[i] × v[ACol[i]];
    i += warpSize
end
__syncthreads();
/* Reduction Phase                    */;
if lane ≡ 0 then
    result[row] = warp_shared[0]);
end
```

TABLE I
SYSTEM HARDWARE SPECIFICATIONS

Device	Metric	Value
NVIDIA A30	Memory BW	933 GB/s
	FP32 Perf.	10.3 TFLOPS
	Threads/Block	1,024
	Shared Memory	164 KB
EPYC 9334	Cores	64
	Threads	128
	Frequency	3.91 GHz

Software

All the programs were run using the following configuration:

- for CPU: *gcc -O3*;
- for GPU: *nvcc -O3 –use_fast_math*.

The version of *gcc* is 11.5.0 and *CUDA* 11.8.0. The data-type used to store the non-zeroes values is Float32.

E. Measurement methodology

Each measurement is obtained by averaging the performance of each algorithm across 50 timed iterations with 5 previous warm-up cycles.

The following metrics were collected:

- **Execution time**, in seconds $[s]$;
- **Bandwidth**, measured as $Used\_GB \div time$ $[GB/s]$;
- **Floating point operations per second** (FLOPS), measured as the standard $\frac{2 \times nnz}{10^9 \times time}$ $[GFLOPS]$.

For reproducibility, the random dense vector has been set to have all values equal to 1.0.

III. DATASET

The experiments were tested against 10 different matrices sourced from SuiteSparse Matrix Collection[3], with different regimes in size and sparsity structure. Table II provides some characterization of the matrices.

TABLE II
OVERVIEW OF THE MATRICES USED

#	Matrix	Dimensions[4]	nnz	nnz/row	Type	Sym.
1	FullChip	2,987,012	26,621,990	8.9	Unstr.	No
2	Ga41As41H72	268,096	18,488,476	69.0	Unstr.	Yes
3	Si41Ge41H72	185,639	15,011,265	80.9	Unstr.	Yes
4	bone010	986,703	47,851,783	48.5	Diag.	Yes
5	ldoor	952,203	42,493,817	44.6	Unstr.	Yes
6	rajat31	4,690,002	20,316,253	4.3	Diag.	No
7	ASIC_680ks	682,712	1,693,767	2.5	Unstr.	No
8	Rucci1	1,977,885 ×109,900	7,901,068	4.0	Unstr.	No
9	boyd2	466,316	1,500,397	3.2	Unstr.	No
10	webbase-1M	1,000,005	3,105,536	3.1	Unstr.	No

IV. RESULTS

CPU Results

The CPU implementations performed as follow: the COO version had an average bandwidth of 9.21 GB/s and 1.35 GFLOPS, while the CSR one had 17.60 GB/s and 2.64 GFLOPS respectively. As expected, the CSR version outperforms the COO one. Fig. 1 shows their performance.

GPU Results

Fig. 2 shows the bandwidth and the FLOPS of the COO while Fig. 3 of the CSR one. Their corresponding average performances, across all matrices and block/thread configurations are of 113.10 GB/s and 16.86 GFLOPS and 2783.39 GB/s and 442.64 GFLOPS.

The corresponding (with sequential access) GPU implementations of the CPU algorithms have an average speed-up of 21.04× for the COO and 10.28× for the CSR across all matrices. The best performing GPU algorithms however have, respectively, a speed-up equal to 22.37× and 228.05×. The worse speed-up is achieved by FullChip (CSR) with a value of 3.33×. Even in the worst case, the GPU is at least 3 times faster compared to the CPU implementation.

Kernel configuration results

Figs. 4 and 5 show the performance according to the number of threads and blocks on the GPU implementations.

V. DISCUSSION

The first thing to note, is how the sequential access pattern performs worse compared to the strided one. As explained al-

[3]https://sparse.tamu.edu/ - Last visited on: May 16, 2026

[4]Only one dimension reported if the matrix is square

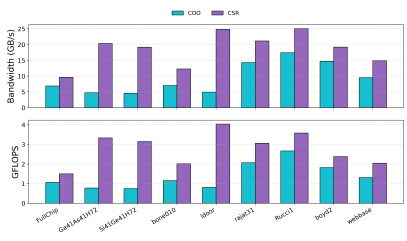

Fig. 1. CPU Performance

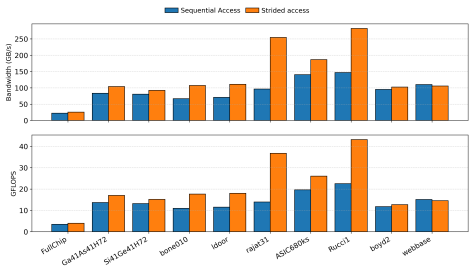

Fig. 2. GPU COO Performance

Fig. 3. GPU CSR Performance

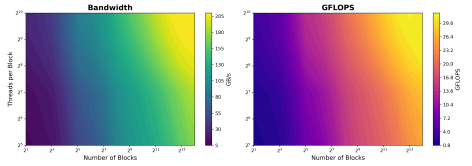

Fig. 4. COO Block and number of threads performance

ready, this is the expected behaviour, due to the improvements on memory coalescing.

It is also worth noting how the CSR configurations have significantly higher average values compared to the COO ones, even exceeding the theoretical peak bandwidth of the GPUs. This is caused by two factors: the caching of the values and the short time of execution of some iterations, reaching as low as 70 microseconds, amplify the inaccuracies of the bandwidth and FLOPS calculations. However, discarding these outliers, we still can see, as expected, how the CSR implementation consistently outperforms the COO one.

Regarding the COO performance, we can see that as the number of threads and blocks increases, the performance, generally, does so as well. This is the expected behaviour. On the other hand, for the CSR format, we see no clear patterns, due to the outliers already stated. However, we have to note that the block number was calculated at run-time, so the number of blocks does not start at 2 like the COO, filtering out obvious worse performant combinations. Regardless, a general trend is visible: higher thread counts tend to improve performance. Best results can be seen in the upper-left region and can be seen worsening along the diagonal, with some exceptions. This suggests that the additional parallelism obtained from increasing the number of blocks eventually becomes limited by memory-system pressure and kernel-management overheads. Note how this behaviour is, as stated, not true for the COO format as the performances improves with the additional number of blocks and threads

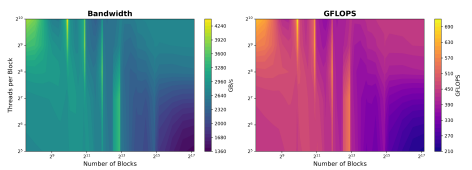

Fig. 5. CSR Block and number of threads performance

By looking with more attention to each matrix performance, we can see how for the COO implementation the best performing matrices are those with a lower average nnz/row and a more diagonal pattern. These two factors together ensure that each thread does not have to work on many values, improving both the performance and the contention on the *atomicAdd* call. However, we can see that the FullChip matrix is performing poorly despite the low nnz/row value. This is due to the distributions of the non-zero values, as they are concentrated, leading to an uneven work distribution, significantly reducing performance. For the CSR formats, we can see that the symmetry plays a big role in the performance. This aids the caching of the values, enabling faster retrieval times. Surprisingly, we can see how the shared memory does not improve the overall performance. This suggests that the synchronization overhead and shared-memory management costs outweigh the benefits of global memory accesses.

VI. Conclusion

The results show both that the CSR performs significantly better than COO due to its reduced memory footprint and the removal of costly *atomicAdd()* operations and that strided access patterns consistently improved performance over a sequential accesses due to better memory coalescing and efficient warp-level memory transactions.

The experiments also confirmed that SpMV is primarily a memory-bound problem. Performance improvements were mainly associated with more efficient memory accesses, higher cache reuse, and improved thread scheduling rather than increased computational throughput. In particular, matrices with regular sparsity patterns and lower average nnz/row generally achieved better performance due to reduced contention and improved load balancing.

Future work could explore additional sparse formats such as ELLPACK or HYB and adaptive kernels specialized for different sparsity structures, but even these simpler implementations of SpMV on GPU which mostly take advantage of only access patterns already show significant improvements compared to the CPU.

References

[1] Genshen Chu, Yuanjie He, Lingyu Dong, Zhezhao Ding, Dandan Chen, He Bai, Xuesong Wang, and Changjun Hu. Efficient algorithm design of optimizing spmv on gpu. In *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, HPDC '23, page 115–128, New York, NY, USA, 2023. Association for Computing Machinery.

[2] Jianhua Gao, Bingjie Liu, Weixing Ji, and Hua Huang. A systematic literature survey of sparse matrix-vector multiplication, 2024.

[3] Samuel Williams, Leonid Oliker, Richard Vuduc, John Shalf, Katherine Yelick, and James Demmel. Optimization of sparse matrix-vector multiplication on emerging multicore platforms. In *SC '07: Proceedings of the 2007 ACM/IEEE Conference on Supercomputing*, pages 1–12, 2007.